

CodLab: Sensor Monitor Pro

Edición Flutter · Guía Maestra: MVVM, Dart 3 y Provider

Proyecto de Ingeniería de Software · 2026

1 Introducción

Este laboratorio está pensado para estudiantes que se inician en Flutter. Construirás, paso a paso, una aplicación multiplataforma que monitorea sensores aplicando los tres pilares profesionales:

- **Arquitectura MVVM:** separa la lógica de datos de la interfaz de usuario.
- **Programación Reactiva:** usa *ChangeNotifier* + *Provider* para que la UI se redibuje automáticamente cuando cambian los datos.
- **Robustez de Datos:** aplica el *null safety* de Dart para evitar cierres inesperados ante fallos de hardware.

Antes de empezar

Necesitas tener instalado el **Flutter SDK** (3.27 o superior, que incluye Dart 3) y un IDE: te recomendamos **Visual Studio Code** con la extensión **Flutter**, aunque Android Studio con su plugin Flutter también funciona. Verifica tu instalación abriendo una terminal y ejecutando `flutter doctor`; todos los *checks* relevantes deben aparecer en verde.

2 Paso 1: Creación del Proyecto

A diferencia de Android nativo, los proyectos Flutter se crean desde la terminal.

1. Abre una terminal y navega a la carpeta donde guardas tus proyectos.
2. Ejecuta el comando:

```
flutter create --org com.illareklab sensor_monitor_pro
```

El parámetro `--org` define el identificador único del paquete (similar al *Package Name* de Android). El nombre del proyecto debe ir en **snake_case** (minúsculas y guiones bajos): es una regla estricta de Dart.

3. Entra a la carpeta del proyecto y ábrela en VS Code:

```
cd sensor_monitor_pro  
code .
```

4. Verifica que todo funcione: pulsa **F5** (o ejecuta `flutter run` en la terminal). Debería arrancar el contador clásico de Flutter en tu emulador o navegador.

Captura sugerida: la app de contador inicial corriendo en el emulador.

Pegue aquí su captura del proyecto Flutter recién creado

Sobre el emulador

Si no tienes un emulador Android, Flutter puede correr la app directamente en **Chrome** (web) o en una **ventana de escritorio** en Windows/macOS/Linux. Para este laboratorio cualquiera funciona; te recomendamos Chrome porque arranca en segundos.

3 Paso 2: Estructura de Carpetas (Capas MVVM)

Flutter no impone una estructura: tú la creas. Toda la lógica vive dentro de `lib/`.

3.1 Crear las carpetas

Dentro de `lib/` crea tres carpetas (clic derecho en `lib` ¿**New Folder**):

- `model/` — clases de datos puros
- `viewmodel/` — lógica de estado y reglas de negocio
- `view/` — pantallas (widgets) que el usuario ve

Estructura final esperada:

```
lib/  
|-- model/  
|-- viewmodel/  
|-- view/  
|-- main.dart (ya existe)
```

Pegue aquí su captura del explorador de archivos con las carpetas creadas

Convención de nombres en Dart

Los archivos Dart se nombran en **snake_case.dart**, no en **CamelCase**. Las clases sí van en **PascalCase**. Por ejemplo: el archivo `sensor_data.dart` contiene la clase `SensorData`.

4 Paso 3: Capa Model (Datos)

Dentro de lib/model/ crea el archivo `sensor_data.dart` (clic derecho en model ¿New File).

```
/// Clase que representa la lectura de un sensor.
/// El campo "value" es double? (anulable) para simular
/// sensores desconectados o con fallo de hardware.
class SensorData {
  final int id;
  final String label;
  final double? value;
  final String unit;

  const SensorData({
    required this.id,
    required this.label,
    required this.value,
    required this.unit,
  });

  /// Devuelve la lectura formateada.
  /// Si value es null, muestra "--" gracias al operador ??
  String getReading() => '${value ?? "--"} $unit';
}
```

¿Qué es el operador ???

`value ?? -"` significa: *si value no es nulo, úsalo; si es nulo, usa -"*. Es el equivalente directo del operador Elvis (`?:`) de Kotlin. En Dart se llama **null-coalescing operator**.

5 Paso 4: Capa ViewModel (Lógica y Estado)

Crearemos dos archivos dentro de lib/viewmodel/.

5.1 4.1 ui_state.dart (Sealed Class)

Define los tres estados posibles que puede tener la pantalla. Usamos las *sealed classes* de Dart 3, que obligan al compilador a verificar que cubrimos todos los casos en el `switch`.

```
import 'package:sensor_monitor_pro/model/sensor_data.dart';

sealed class UIState {
  const UIState();
}

class Loading extends UIState {
  const Loading();
}

class Success extends UIState {
  final List<SensorData> sensors;
  const Success(this.sensors);
}

class Failure extends UIState {
  final String message;
  const Failure(this.message);
}
```

¿Por qué Failure y no Error?

Error ya existe en la librería estándar de Dart (`dart:core`) y reservarlo causa confusión y avisos del analizador. Usamos `Failure` para evitar el choque de nombres.

5.2 4.2 sensor_view_model.dart

Es el cerebro de la aplicación. Carga los datos de forma asíncrona y notifica a la UI cuando cambian.

```
import 'package:flutter/foundation.dart';
import 'package:sensor_monitor_pro/model/sensor_data.dart';
import 'package:sensor_monitor_pro/viewmodel/ui_state.dart';

class SensorViewModel extends ChangeNotifier {
  // Estado privado: solo el ViewModel puede modificarlo
  UIState _uiState = const Loading();

  // Getter publico de SOLO LECTURA hacia la UI
  UIState get uiState => _uiState;

  SensorViewModel() {
    _loadData();
  }

  Future<void> _loadData() async {
    // Simula la lectura de un hardware real
    await Future.delayed(const Duration(seconds: 2));

    _uiState = Success([
      const SensorData(id: 1, label: 'Temperatura', value: 24.5, unit: 'C'),
      const SensorData(id: 2, label: 'Humedad', value: null, unit: '%'),
    ]);

    // Notifica a todos los widgets que escuchan: redibujense
    notifyListeners();
  }
}
```

¿Por qué _uiState privado y uiState público?

Es el patrón de **encapsulación**. En Dart, todo identificador que empieza con guion bajo (`_`) es *privado al archivo*. La versión privada se puede modificar dentro del ViewModel; la pública es de solo lectura, así garantizamos que la *Vista* no toque el estado por accidente.

6 Paso 5: Capa View (Interfaz de Usuario)

Esta es la pieza que convierte el estado en pantalla. Antes de escribir nada de código, hay que agregar el paquete `provider`.

6.1 5.1 Agregar la dependencia provider

`ChangeNotifier` viene incluido en Flutter, pero el widget que lo *escucha* (`ChangeNotifierProvider` y `Consumer`) vive en un paquete aparte llamado `provider`. Sin él, la UI no se redibujará automáticamente.

1. Abre la terminal integrada de VS Code (`Ctrl + ``) en la raíz del proyecto.

2. Ejecuta:

```
flutter pub add provider
```

Este comando agrega la línea correspondiente al `pubspec.yaml` y descarga el paquete automáticamente.

3. Verifica que tu `pubspec.yaml` contiene ahora algo como esto en la sección `dependencies`:

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  provider: ^6.1.2    # <-- la version puede variar
```

Pegue aquí su captura del `pubspec.yaml` con `provider` agregado

Sobre el `^` en las versiones

El símbolo `^6.1.2` significa *cualquier versión $\geq 6.1.2$ y $< 7.0.0$* . Es la convención *semantic versioning* de `pub.dev`.

6.2 5.2 Crear `sensor_screen.dart`

Dentro de `lib/view/` crea el archivo `sensor_screen.dart`.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:sensor_monitor_pro/viewmodel/sensor_view_model.dart';
import 'package:sensor_monitor_pro/viewmodel/ui_state.dart';

class SensorScreen extends StatelessWidget {
  const SensorScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Sensor Monitor Pro'),
        backgroundColor: Theme.of(context).colorScheme.primaryContainer,
      ),
      body: Consumer<SensorViewModel>(
        builder: (context, vm, _) {
          // Pattern matching de Dart 3 sobre la sealed class
          return switch (vm.uiState) {
            Loading() => const Center(
              child: CircularProgressIndicator(),
            ),
            Failure(message: final msg) => Center(
              child: Text(
```



```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (_) => SensorViewModel(),
      child: MaterialApp(
        title: 'Sensor Monitor Pro',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          useMaterial3: true,
          colorSchemeSeed: const Color(0xFFDA8F81),
        ),
        home: const SensorScreen(),
      ),
    );
  }
}
```

Recordatorio rápido si algo falla aquí

Si Dart marca Target of URI doesn't exist: 'package:provider/provider.dart', regresa al **Paso 5.1**: te falta ejecutar `flutter pub add provider`, o no esperaste a que terminara la descarga.

7 Paso 6: Compilación y Resultados Esperados

Pulsa **F5** en VS Code (o ejecuta `flutter run` en la terminal). Deberías observar dos estados consecutivos:

1. **Estado Loading**: durante los primeros 2 segundos aparece un círculo de carga centrado bajo la barra superior.
2. **Estado Success**: aparecen dos tarjetas:
 - **Temperatura**: 24.5 C
 - **Humedad**: --% (porque su valor es null)

Captura sugerida 1: estado de carga (los primeros 2 segundos).

Pegue aquí su captura del estado Loading

Captura sugerida 2: estado final con los dos sensores.



Pegue aquí su captura del estado Success

Hot Reload: el superpoder de Flutter

Mientras la app corre, guarda cualquier cambio en el código y Flutter actualizará la UI **en menos de un segundo** sin reiniciar el estado. Pruébalo: cambia el texto del **AppBar** y guarda. No es magia, es *hot reload*.

8 Conclusión

¡Felicidades! Has construido tu primera aplicación Flutter con arquitectura MVVM real. Repasa lo que lograste:

- **Model:** clase `SensorData` con campos anulables y método de formateo.
- **ViewModel:** `ChangeNotifier` con estado privado, getter público y notificación reactiva.
- **View:** `StatelessWidget` que reacciona automáticamente a los cambios vía `Consumer`.
- **Sealed class + switch expression:** estados exhaustivos verificados en compilación.
- **Null Safety:** el operador `??` protegió la app de un crash.

Reto opcional

Agrega un tercer sensor (*p. ej.* "Presión") y un `FloatingActionButton` con un ícono de refrescar que vuelva a poner el estado en `Loading` y recargue los datos tras un `Future.delayed`. Pista: añade un método público `refresh()` en el `ViewModel` que reasigne `_uiState = const Loading()`, llame `notifyListeners()` y luego invoque `_loadData()`.